	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 3

Integrante(s): 120002850 323064039

323071378 323214636

426059576 426058744

Semestre: 2027-1

Fecha de entrega: 11-Septiembre-2026

Obervaciones:

CALIFICACIÓN:

Índice

1. Introducción **3**

1.1. Planteamiento del problema 3

1.2. Motivación 3

1.3. Objetivos 3

2. Marco Teórico **3**

3. Desarrollo **5**

3.1. Investigación y análisis de la consulta 5

3.2. Interfaz gráfica de login 8

3.3. Contrato de huella de carbono 10

3.4. Clase Edificio 11

3.5. Clase Auto 12

3.6. Clase Bicicleta 14

3.7. Clase principal y polimorfismo 15

4. Resultados **17**

4.1. Casos y resultados numéricos 17

4.2. Ejecución desde clases y desde JAR 19

4.3. Documentación y empaquetado 20

5. Conclusiones **21**

1. Introducción

1.1. Planteamiento del problema

La práctica solicita resolver dos necesidades relacionadas con la programación orientada a objetos. La primera consiste en plantear una interfaz gráfica elemental para capturar un usuario y una contraseña. La segunda requiere representar tres entidades distintas —un edificio, un automóvil y una bicicleta— mediante clases que implementen una misma interfaz llamada `HuellaCarbono`. Cada objeto debe calcular su huella con los datos propios de su dominio y la clase principal debe recorrer una lista de referencias comunes para demostrar polimorfismo.

1.2. Motivación

El ejercicio permite observar que una interfaz define un contrato sin obligar a que todas las clases compartan la misma fórmula. El edificio relaciona consumo eléctrico y factor de emisión; el automóvil relaciona distancia, rendimiento y combustible; la bicicleta expresa que su operación no produce emisiones directas en el modelo didáctico. La GUI complementa el aprendizaje al mostrar una interacción visual sencilla con componentes de Swing y un campo especializado para contraseñas.

1.3. Objetivos

El objetivo general es implementar y comprobar una solución básica en Java que use una interfaz, implementaciones concretas y polimorfismo. Como objetivos específicos se establecen: crear las clases `Edificio`, `Auto` y `Bicicleta`; declarar el método `getHuellaCarbono()`; almacenar las instancias en un `ArrayList<HuellaCarbono>`; presentar una ventana de login con usuario y contraseña; documentar cada clase; y comparar cálculos esperados con salidas observadas en ejecución.

2. Marco Teórico

Una interfaz en Java declara un contrato de operaciones que una clase concreta debe cumplir. La especificación del lenguaje describe que una clase puede implementar una o varias interfaces y que una referencia al tipo de la interfaz puede utilizarse para invocar operaciones definidas por ese contrato [1]. En esta práctica, `HuellaCarbono` solo declara `getHuellaCarbono()`, mientras que cada clase determina la forma de obtener el valor. La selección de la implementación ocurre durante la ejecución, lo que constituye el polimorfismo utilizado por el recorrido de la lista.

El encapsulamiento concentra los datos de cada entidad dentro de la clase que conoce su significado. Por ello, los consumos, distancias y factores se mantienen como atributos privados, mientras que los métodos públicos ofrecen consultas y el cálculo requerido. La solución evita herencia porque las tres entidades no forman una jerarquía natural: comparten el contrato de huella, pero no una estructura de estado común.

La interfaz gráfica se construye con Swing. Un `JFrame` funciona como ventana; `JTextField` captura el usuario; `JPasswordField` proporciona un componente apropiado para una contraseña; `JButton` inicia la acción; y `JLabel` comunica el resultado. La documentación de Oracle presenta estos componentes como parte de la biblioteca de interfaces gráficas de Java y recomienda asociar acciones a los controles mediante listeners [2].

La huella de carbono depende del alcance y de los factores seleccionados. Los inventarios oficiales separan fuentes y unidades, por lo que un factor eléctrico debe interpretarse según la región y el periodo analizado [3]. Para mantener el ejercicio reproducible se emplean dos factores didácticos explícitos: 0.40 kg CO_2e/kWh para electricidad y 2.31 kg CO_2e/L para gasolina. No se presentan como valores universales; solo permiten comprobar una transformación aritmética estable. El factor de gasolina se redondea a partir de factores de combustión publicados para inventarios de gases de efecto invernadero [4].

3. Desarrollo

3.1. Investigación y análisis de la consulta

La consulta común se redactó para solicitar una explicación de 90 a 120 palabras sobre dos temas del enunciado: una GUI de login con Swing y el diseño de la interfaz `HuellaCarbono`. El archivo `doc/Capturas/query.txt` contiene 104 palabras y conserva los ejemplos numéricos usados en la implementación. No se recibieron las seis respuestas originales ni sus imágenes; por ello, el reporte incluye una síntesis técnica propia y reserva honestamente los seis espacios de evidencia sin atribuir respuestas a personas concretas.

La primera reserva corresponde a Alan, cuyo avatar de referencia es CJ de GTA y cuyo color de guion es `#166A8F`.

Espacio reservado: ../Capturas/1.png

Evidencia pendiente de aportación por el integrante.

Figura 3.1: Reserva de investigación 1: Alan; no se aportó captura original.

La segunda reserva corresponde a Zoé, identificada en la tabla operativa por Snoopy, llamado Snoppy por el equipo, y el color `#A52F64`.

Espacio reservado: ../Capturas/2.png

Evidencia pendiente de aportación por el integrante.

Figura 3.2: Reserva de investigación 2: Zoé; no se aportó captura original.

La tercera reserva corresponde a Luis, con Rabbit como avatar y el color `#9A4C16`; no se añade una franquicia no proporcionada.

Espacio reservado: ../Capturas/3.png
Evidencia pendiente de aportación por el integrante.

Figura 3.3: Reserva de investigación 3: Luis; no se aportó captura original.

La cuarta reserva corresponde a Joseph, con Rick Sanchez como avatar y el color `#197349`.

Espacio reservado: ../Capturas/4.png
Evidencia pendiente de aportación por el integrante.

Figura 3.4: Reserva de investigación 4: Joseph; no se aportó captura original.

La quinta reserva corresponde a Hanniel, con el cubo de Geometry Dash como avatar y el color `#5941A9`.

Espacio reservado: ../Capturas/5.png
Evidencia pendiente de aportación por el integrante.

Figura 3.5: Reserva de investigación 5: Hanniel; no se aportó captura original.

La sexta reserva corresponde a Alex, con Phineas de Phineas y Ferb como avatar y el color `#67531C`.

Espacio reservado: ../Capturas/6.png

Evidencia pendiente de aportación por el integrante.

Figura 3.6: Reserva de investigación 6: Alex; no se aportó captura original.

3.2. Interfaz gráfica de login

La clase `LoginGUI` extiende `JFrame` e implementa `ActionListener`. Su formulario contiene un campo de usuario, un campo de contraseña, un botón y una etiqueta de estado. La validación es deliberadamente pequeña y demostrativa: compara con el usuario `usuario` y la contraseña `poo8`. Esta decisión permite evidenciar el flujo de entrada, validación y respuesta sin convertir el ejercicio en un sistema de autenticación real.

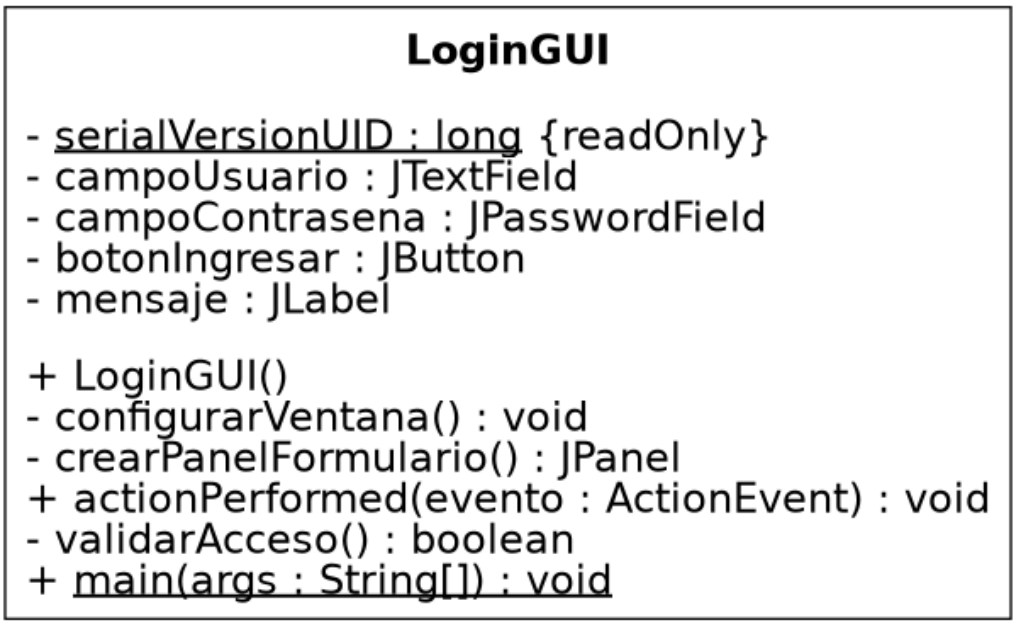


Figura 3.7: Diagrama UML de la clase LoginGUI.

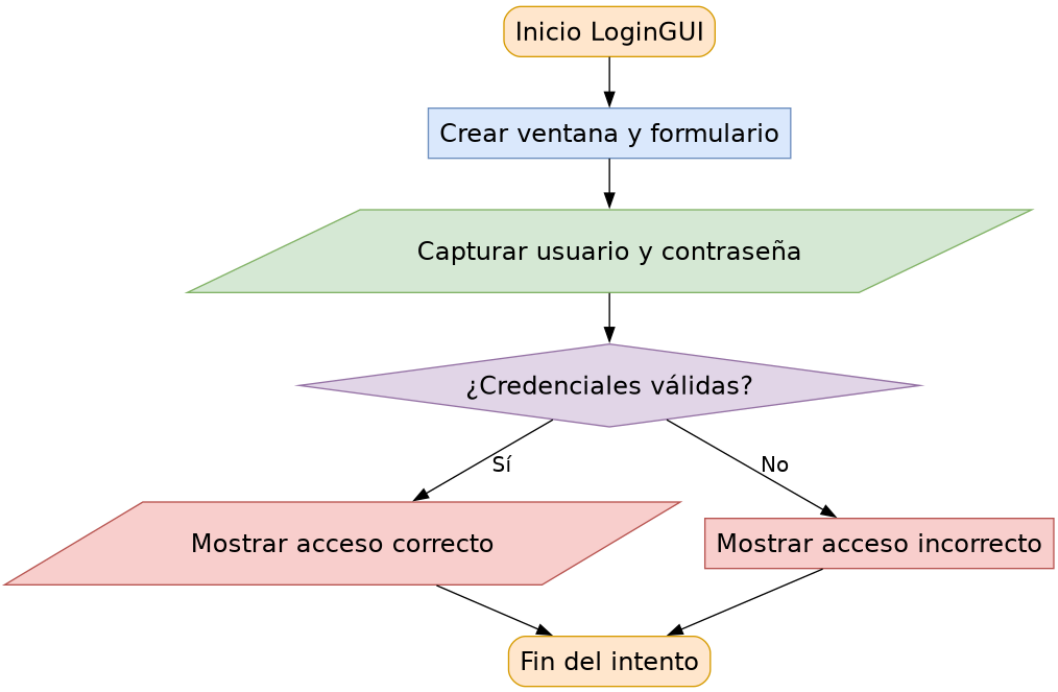


Figura 3.8: Flujo de captura y validación del login.

3.3. Contrato de huella de carbono

La interfaz `HuellaCarbono` contiene un solo método: `double getHuellaCarbono()`. Al mantener el contrato mínimo, el programa principal puede trabajar con referencias del tipo de la interfaz sin preguntar qué clase concreta calcula el valor. El diagrama UML conserva el compartimento de atributos vacío porque la interfaz no declara estado, y el flujo representa la delegación hacia la clase concreta.

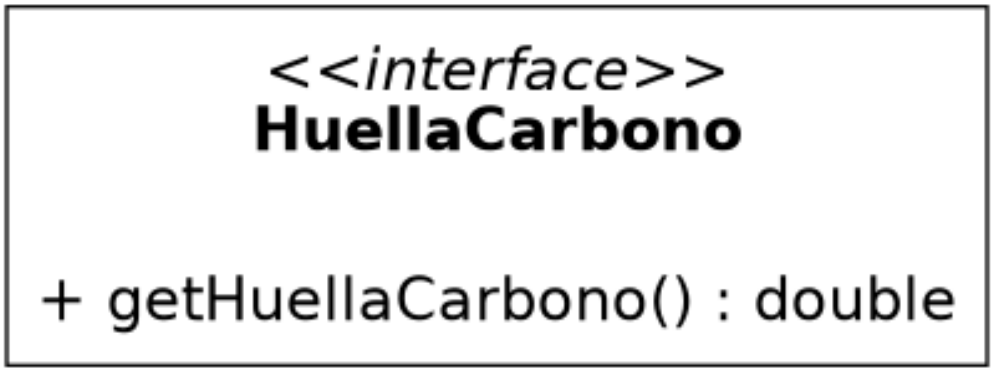


Figura 3.9: Diagrama UML de la interfaz HuellaCarbono.

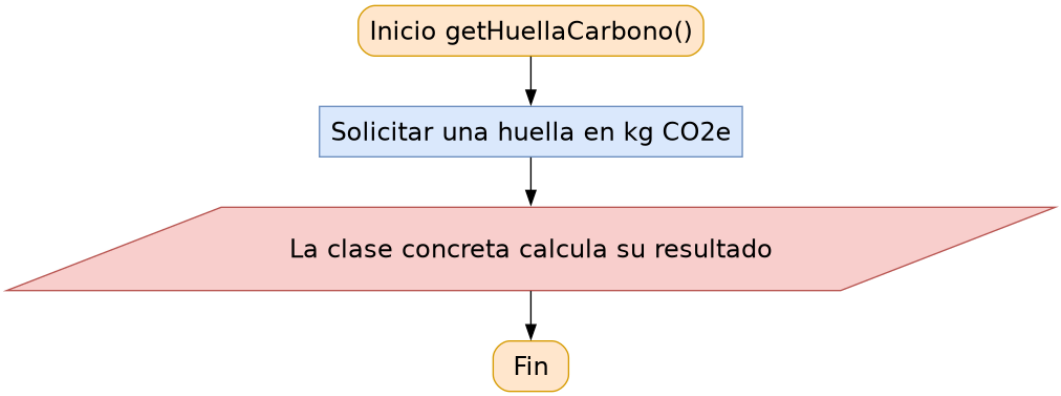


Figura 3.10: Flujo conceptual de la operación getHuellaCarbono.

3.4. Clase Edificio

`Edificio` identifica una construcción por su nombre, consumo anual de electricidad y factor de emisión. La fórmula implementada es $H = C \times F$, donde C se mide en kilowatt-hora y F en kilogramos de CO_2e por kilowatt-hora. El comportamiento es exclusivo de esta clase porque las otras entidades no utilizan un consumo eléctrico anual.

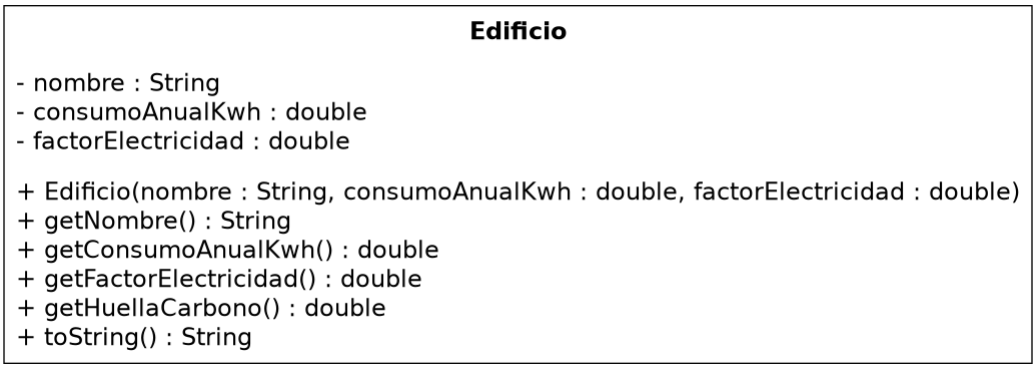


Figura 3.11: Diagrama UML de la clase Edificio.

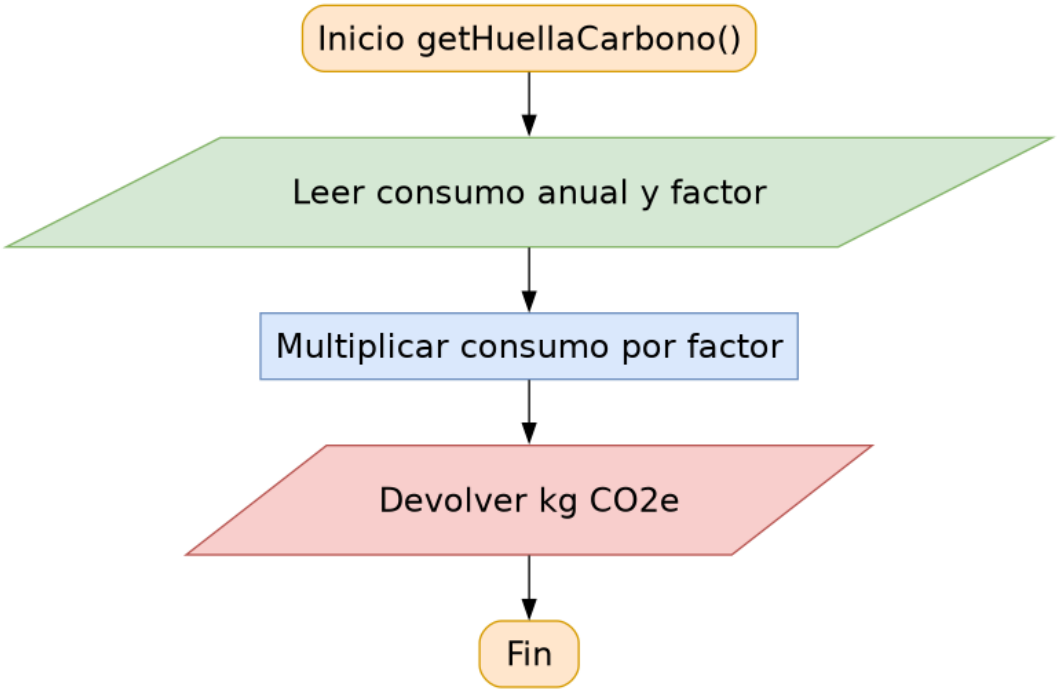


Figura 3.12: Flujo del cálculo de huella de Edificio.

3.5. Clase Auto

`Auto` identifica un modelo y calcula el combustible utilizado con $L = (K \times R)/100$, donde K es la distancia en kilómetros y R es el rendimiento expresado en litros por cada cien kilómetros. Después obtiene la huella con $H = L \times F_g$, usando el factor de gasolina F_g. La separación en dos operaciones hace visible la conversión intermedia y facilita comprobar el resultado.

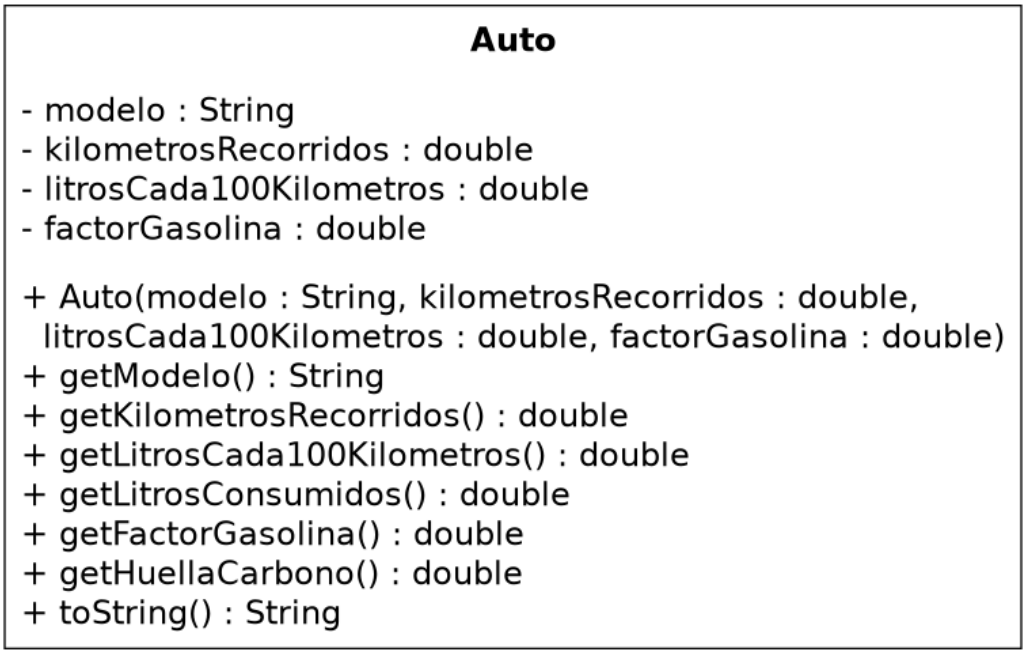


Figura 3.13: Diagrama UML de la clase Auto.

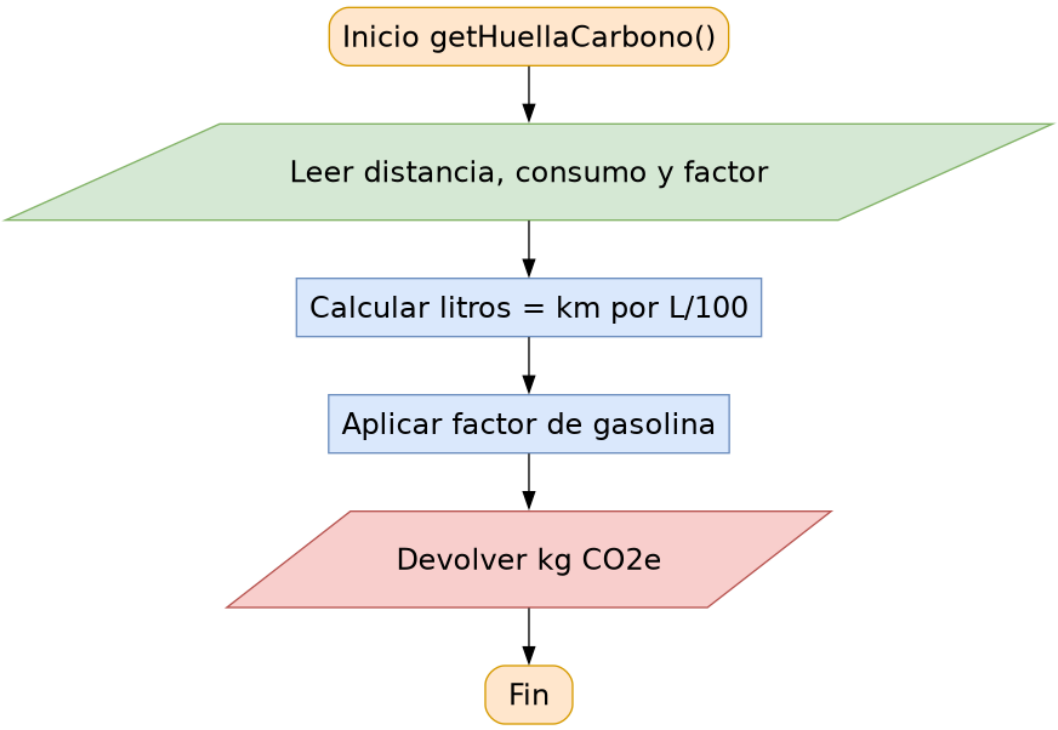


Figura 3.14: Flujo del cálculo de huella de Auto.

3.6. Clase Bicicleta

`Bicicleta` conserva un tipo y una distancia recorrida. El alcance del modelo se limita a emisiones operativas directas; por ello, el factor se fija en cero y la fórmula $H = K \times 0$ devuelve cero. No se afirma que la fabricación, alimentación, mantenimiento o transporte sean nulos, porque esos aspectos no forman parte del enunciado ni del cálculo implementado.

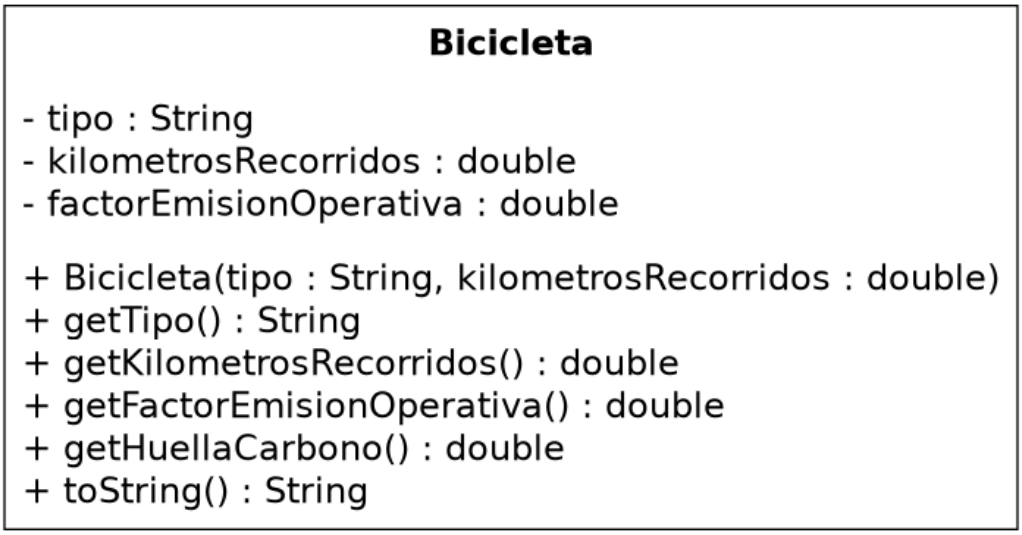


Figura 3.15: Diagrama UML de la clase Bicicleta.

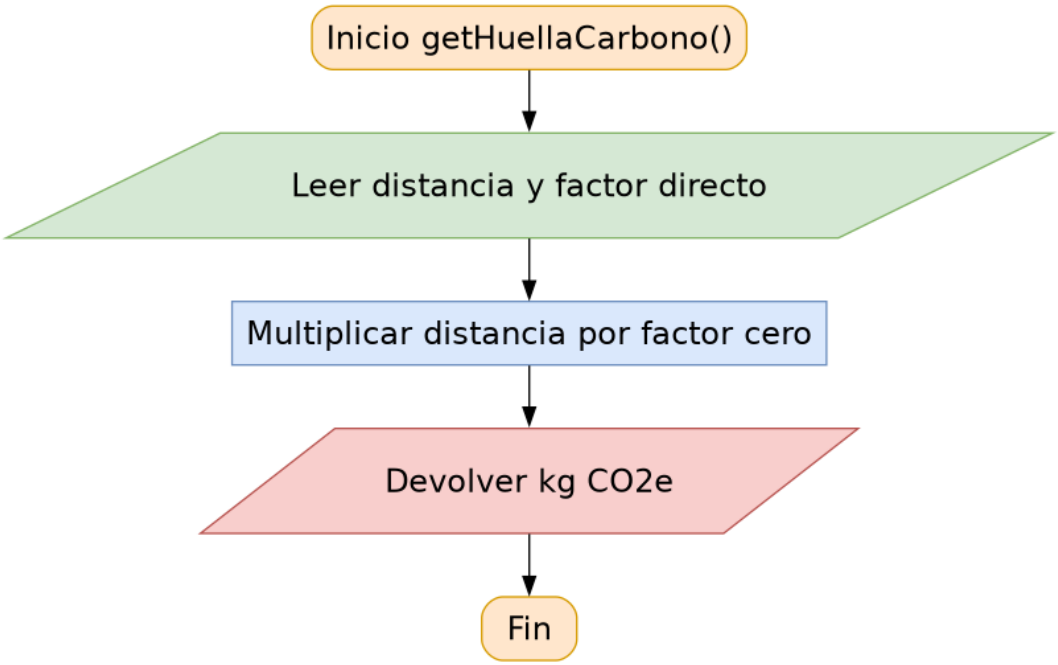


Figura 3.16: Flujo del cálculo de huella operativa de Bicicleta.

3.7. Clase principal y polimorfismo

La clase `Main` crea las tres instancias exigidas y agrega también un edificio con consumo cero como frontera pertinente. Las cuatro referencias se guardan en un `ArrayList<HuellaCarbono>`. El ciclo solo conoce el contrato de la interfaz: imprime la identificación del objeto y llama a `getHuellaCarbono()`. El despacho polimórfico selecciona la fórmula de `Edificio`, `Auto` o `Bicicleta` sin condicionales por tipo.

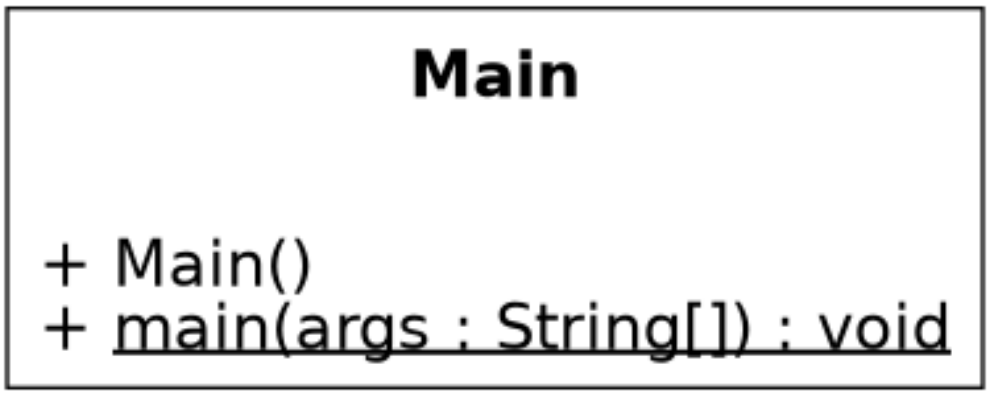


Figura 3.17: Diagrama UML de la clase principal Main.

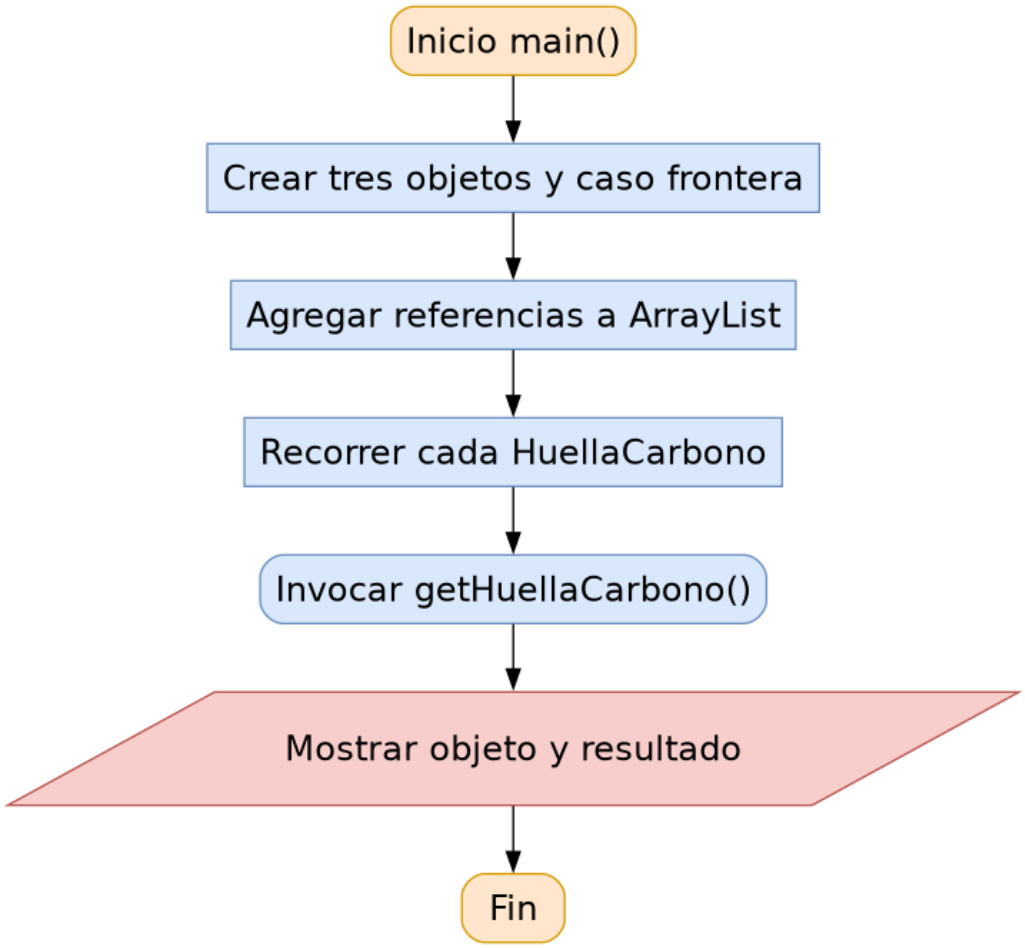


Figura 3.18: Flujo de creación, recorrido e invocación polimórfica.

4. Resultados

4.1. Casos y resultados numéricos

Los resultados esperados se calcularon antes de ejecutar el programa con las fórmulas descritas en el desarrollo. El edificio principal usa $2400 \times 0.40 = 960.00$ kg CO_2e. El automóvil usa $480 \times 7/100 = 33.60$ litros y después $33.60 \times 2.31 = 77.62$ kg CO_2e, redondeado a dos decimales. La bicicleta produce cero emisiones operativas directas en el modelo. Finalmente, el edificio frontera permite comprobar que un consumo nulo no introduce una huella espuria.

Tabla 1: Comparación entre el esperado calculado y el observado ejecutado.

Objeto	**Entrada principal**	**Esperado**	**Observado**	**Estado**
Edificio Escuela	2400 kWh, factor 0.40	960.00	960.00	Exitoso
Auto Sedán	480 km, 7.0 L/100 km, factor 2.31	77.62	77.62	Exitoso
Bicicleta urbana	12 km, factor operativo 0.00	0.00	0.00	Exitoso
Edificio frontera	0 kWh, factor 0.40	0.00	0.00	Exitoso

La ejecución observada también confirma la identificación de cada instancia y el recorrido de la lista mediante la interfaz. No se comparó contra una salida inventada: las cifras anteriores se obtuvieron de la salida real de la clase compilada y del JAR, cuyos registros visuales se incluyen en las figuras siguientes.

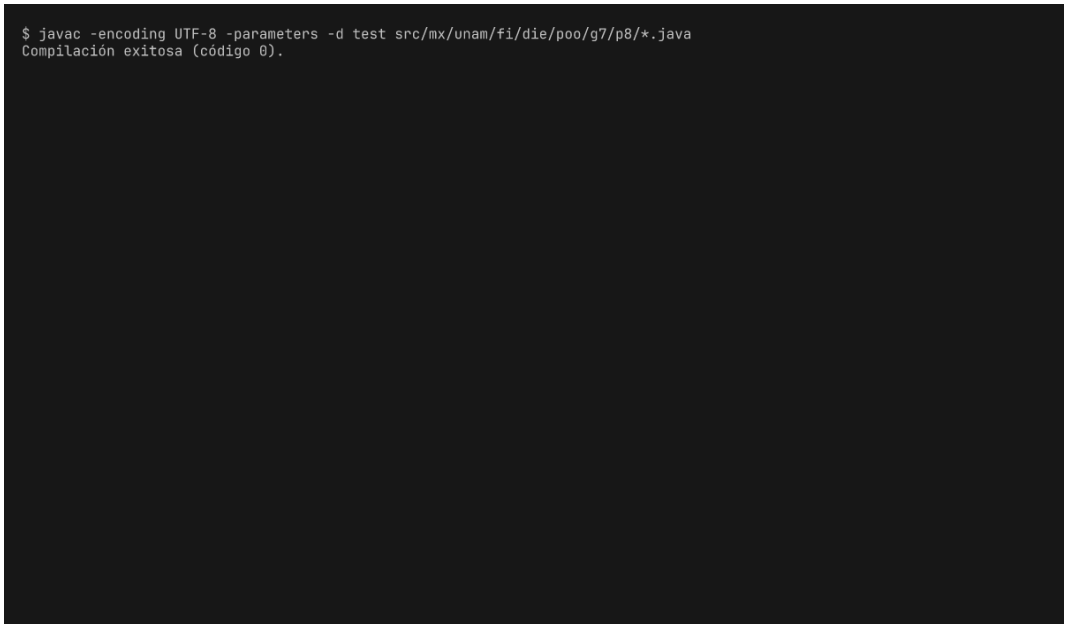


Figura 4.1: Compilación limpia de las seis fuentes Java hacia la carpeta test.

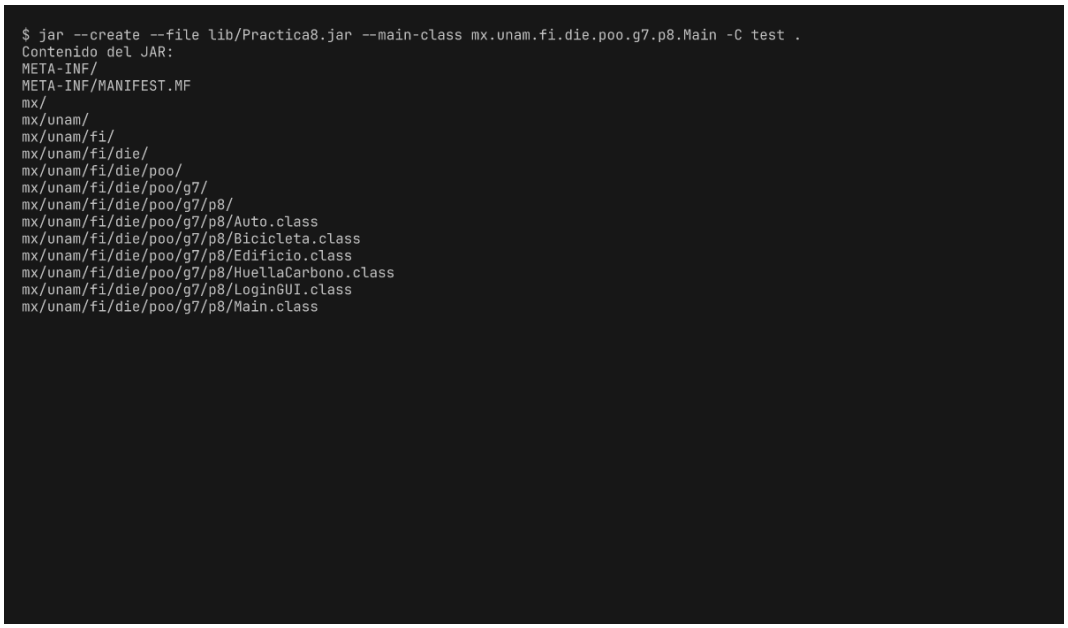


Figura 4.2: Creación y contenido del `Practica8.jar`.

4.2. Ejecución desde clases y desde JAR

La clase principal se ejecutó directamente desde la carpeta de compilados y posteriormente desde el archivo JAR con manifiesto de entrada. Ambas modalidades produjeron las mismas cuatro huellas: 960.00, 77.62, 0.00 y 0.00 kg CO_2e. El JAR contiene `Main.class`, la interfaz, las tres implementaciones de huella y la clase de GUI, por lo que no incorpora clases de prácticas anteriores.

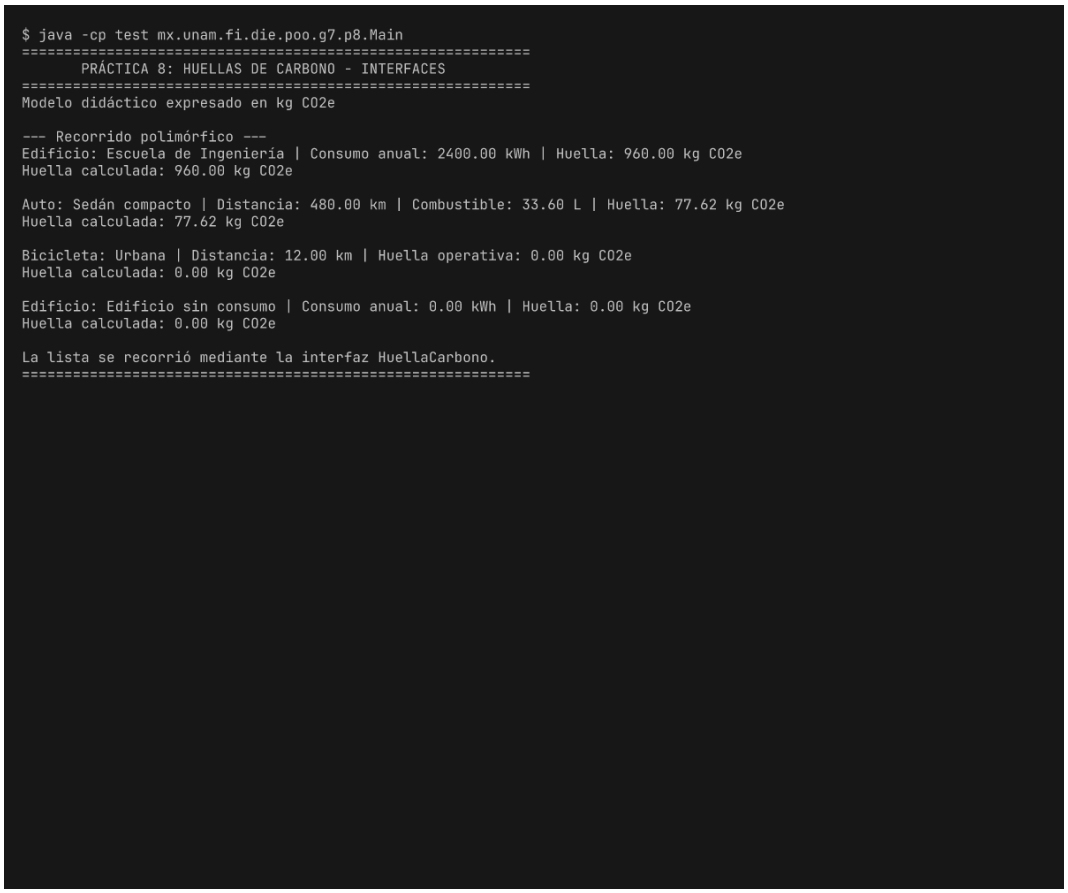


Figura 4.3: Salida observada al ejecutar Main desde la carpeta de compilados.

La compilación de `LoginGUI` fue exitosa y la documentación se generó de forma independiente. No se declara una ejecución visual observada de la ventana porque el entorno de trabajo no dispone de un servidor gráfico; la verificación disponible para este componente es compilación, firmas, Javadoc y revisión estática del flujo. Esta limitación no afecta la ejecución determinista de la parte de consola.

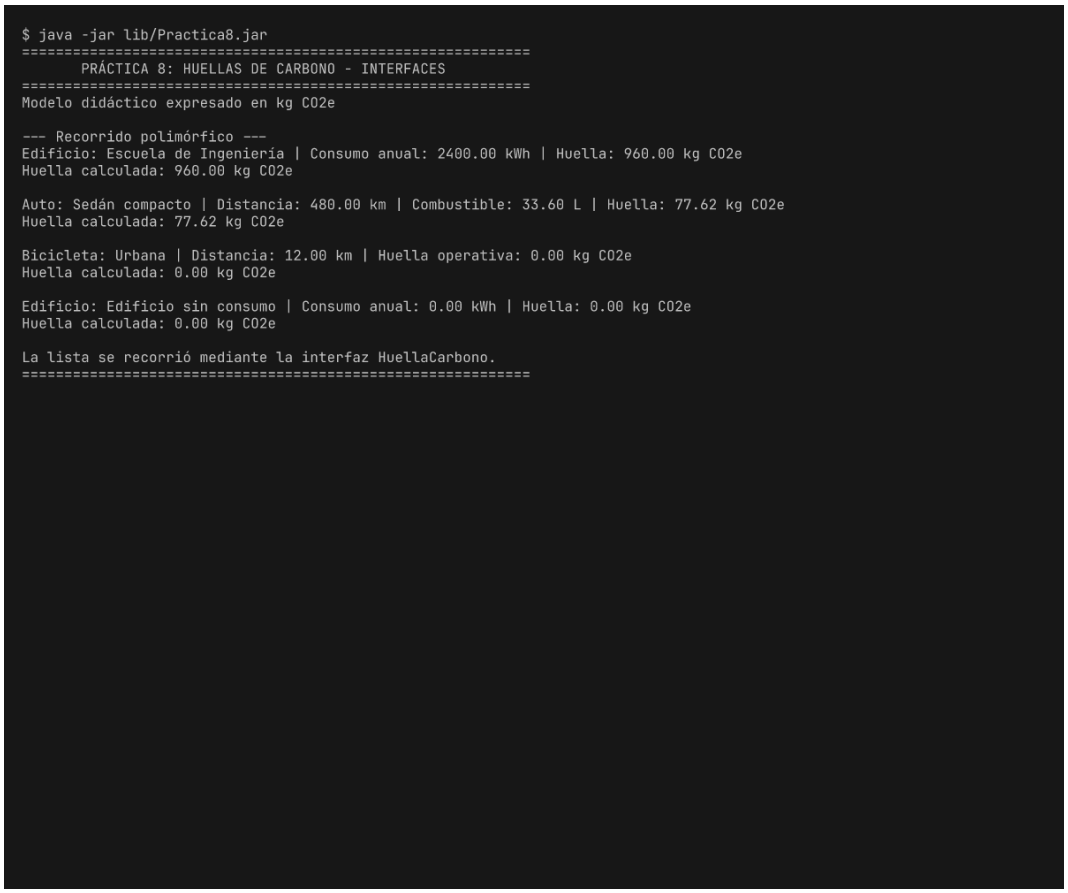


Figura 4.4: Salida observada al ejecutar el trabajo completo desde el JAR.

4.3. Documentación y empaquetado

Se generó una salida independiente de Javadoc para `HuellaCarbono`, `Edificio`, `Auto`, `Bicicleta`, `Main` y `LoginGUI`, cada una en su directorio bajo `doc/Javadoc`. Las invocaciones usaron la fuente explícita de una sola clase, visibilidad privada, codificación UTF-8, classpath de compilados y un sourcepath temporal vacío. También se verificó que `test/` contiene solo los compilados finales y que `lib/` contiene un solo archivo JAR del trabajo.

5. Conclusiones

La interfaz `HuellaCarbono` permitió unificar tres comportamientos que no comparten una jerarquía de datos. Cada clase conserva sus atributos y su fórmula, mientras que `Main` utiliza una referencia común y una lista homogénea. Esta separación muestra una aplicación concreta del polimorfismo: el código que recorre la colección no necesita conocer la clase exacta de cada elemento.

Los resultados numéricos coincidieron con los esperados calculados antes de ejecutar. La frontera de consumo cero confirmó el comportamiento mínimo del modelo, y las ejecuciones desde classpath y JAR conservaron la misma salida. Los factores de emisión se mantuvieron como supuestos didácticos explícitos, porque una evaluación real tendría que seleccionar factores apropiados para una región, periodo y alcance definidos.

La GUI de login completó el flujo académico de captura, validación y respuesta mediante componentes básicos de Swing. Su validación no pretende sustituir autenticación segura; su función es demostrar la interacción gráfica solicitada. En conjunto, la práctica relaciona contrato, implementaciones concretas, documentación y empaquetado reproducible sin agregar una arquitectura innecesaria.

Bibliografía

[1] Oracle, *The Java Language Specification, Java SE 25 Edition: Interfaces*, Oracle Corporation, 2025.

[2] Oracle, *Creating a GUI With Swing*, The Java Tutorials, Oracle Corporation.

[3] U.S. Environmental Protection Agency, *Emissions & Generation Resource Integrated Database (eGRID)*, EPA, 2024.

[4] U.S. Environmental Protection Agency, *Emission Factors for Greenhouse Gas Inventories*, EPA, 2024.